Assignment Title: SQL Sales Data Analysis Using MySQL Workbench

Name: Rohit Balasaheb Kokare

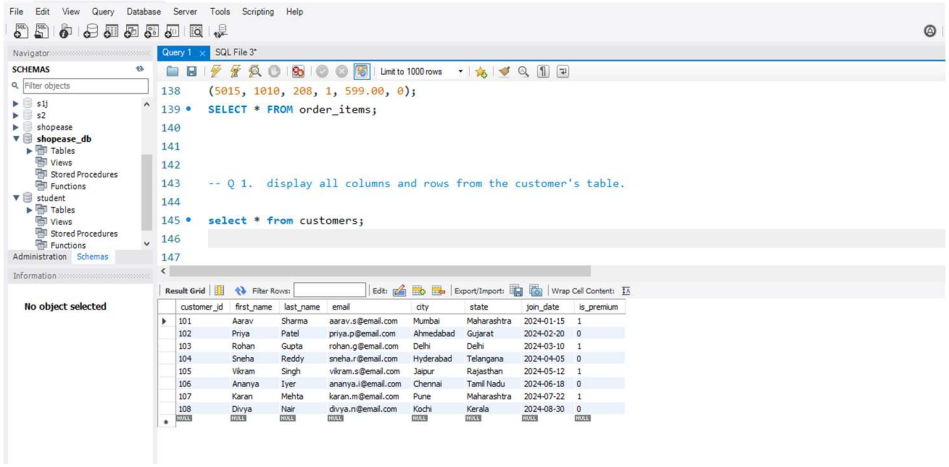
Class: B.Tech (Information Technology)

Roll No.: 83

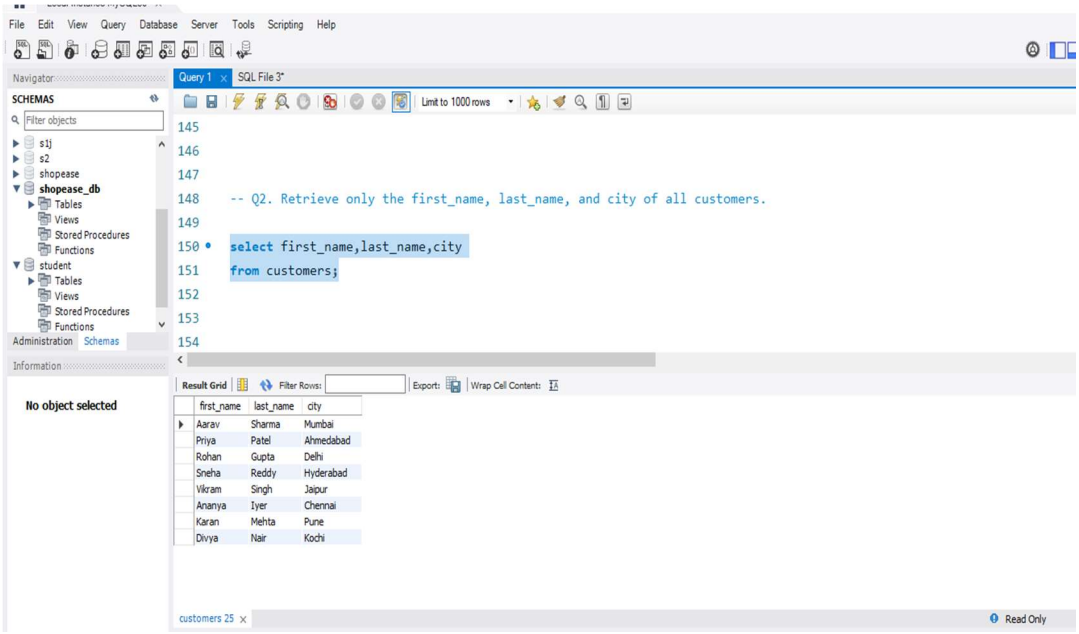
PRN No.:UIT23M1083

Section A — SQL Basics

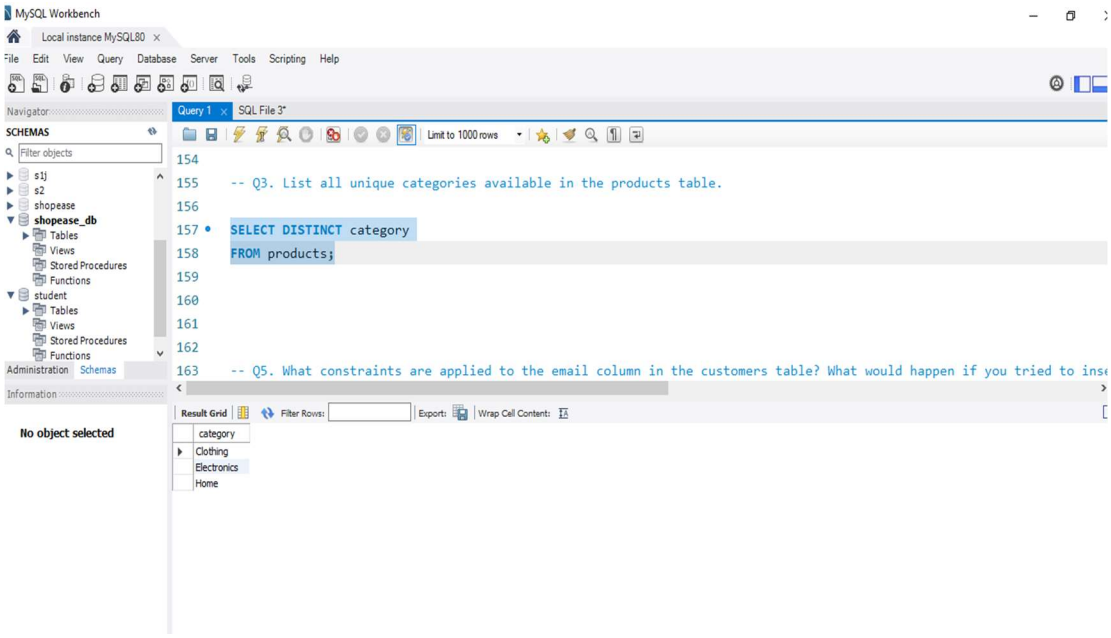
Q1. Write a query to display all columns and rows from the customer's table.



Q2. Retrieve only the first_name, last_name, and city of all customers.



Q3. List all unique categories available in the products table.



Q4. Identify the Primary Key of each table in the schema. Explain why a Primary Key must be unique and NOT NULL.
Primary Keys:

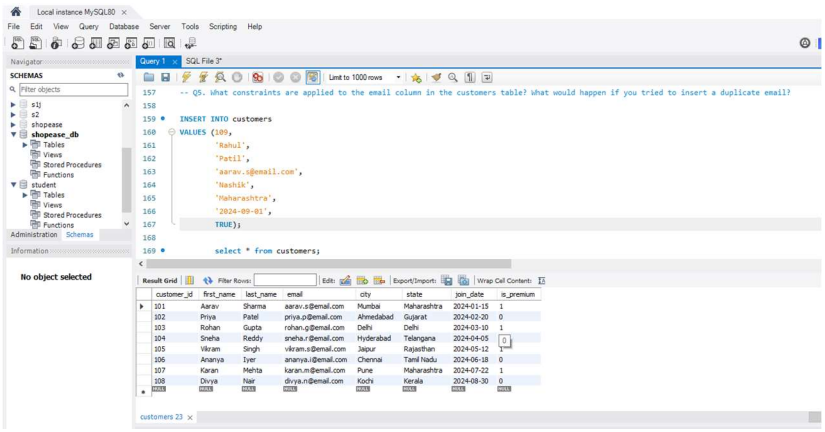
Table	Primary Key
customers	customer_id
products	product_id
orders	order_id
order_items	item_id

Explanation:
A **Primary Key** is a column (or set of columns) that uniquely identifies each record in a table.

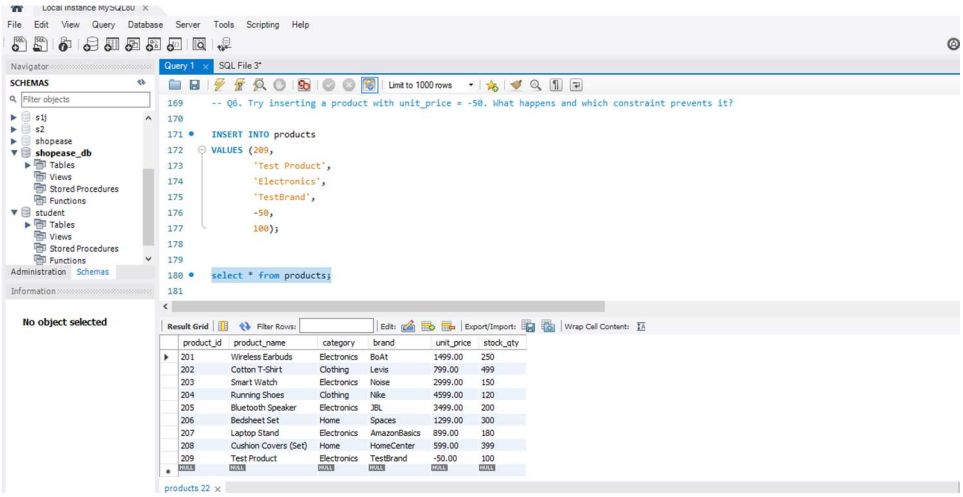
A Primary Key must be:

- **Unique:** No two rows can have the same primary key value.
- **NOT NULL:** Every row must have a value for the primary key.

Q5. What constraints are applied to the email column in the customers table? What would happen if you tried to insert a duplicate email?

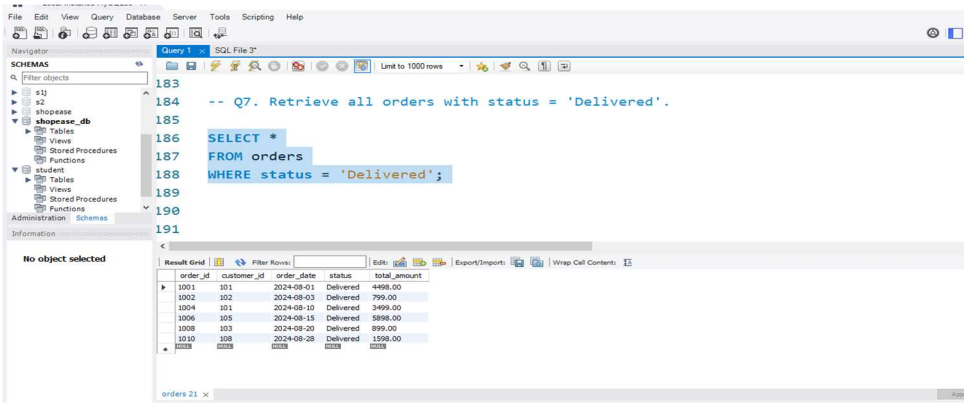


Q6. Try inserting a product with unit_price = -50. What happens and which constraint prevents it? Write both the INSERT statement and explain the error.

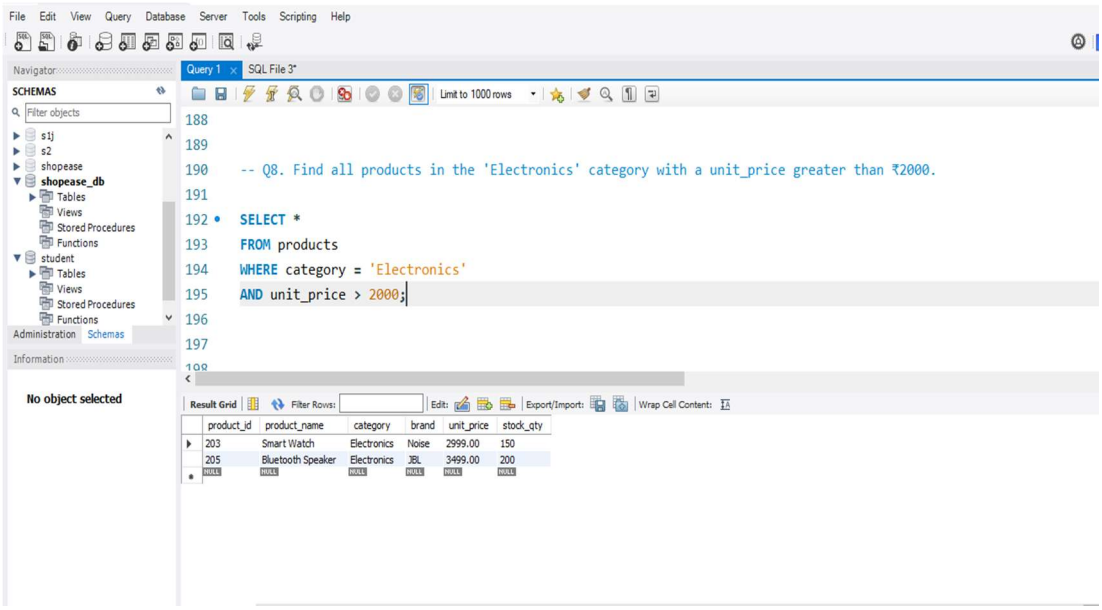


Section B — Filtering & Optimization (WHERE, Indexes)

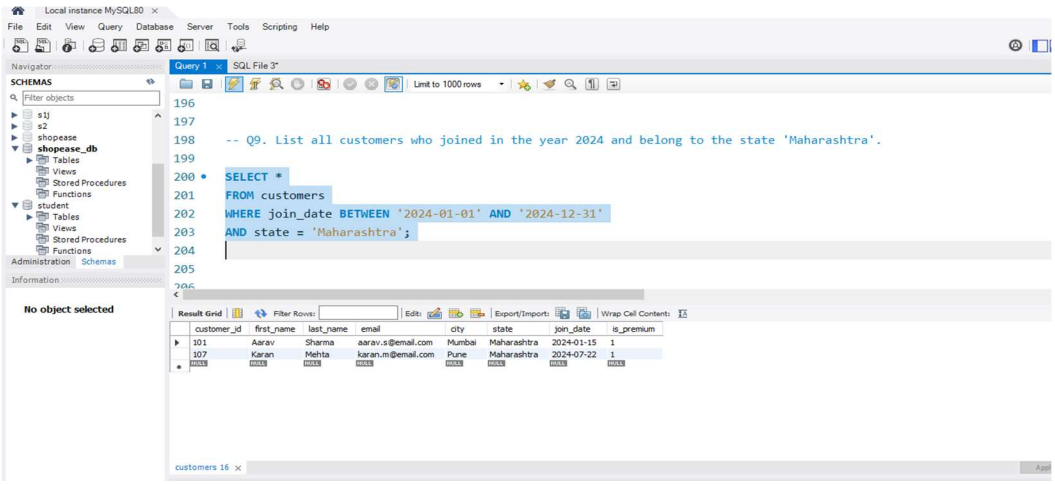
Q7. Retrieve all orders with status = 'Delivered'.



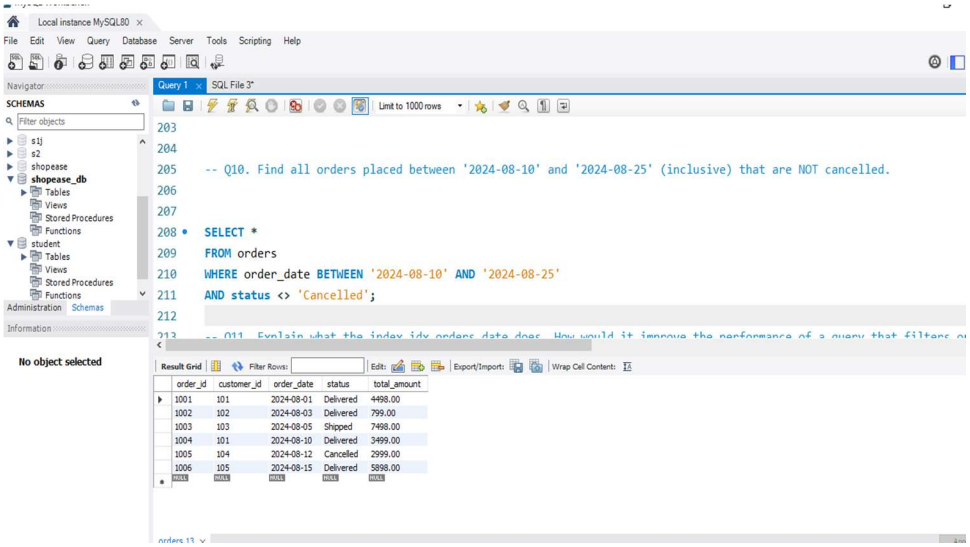
Q8. Find all products in the 'Electronics' category with a unit_price greater than ₹2000.



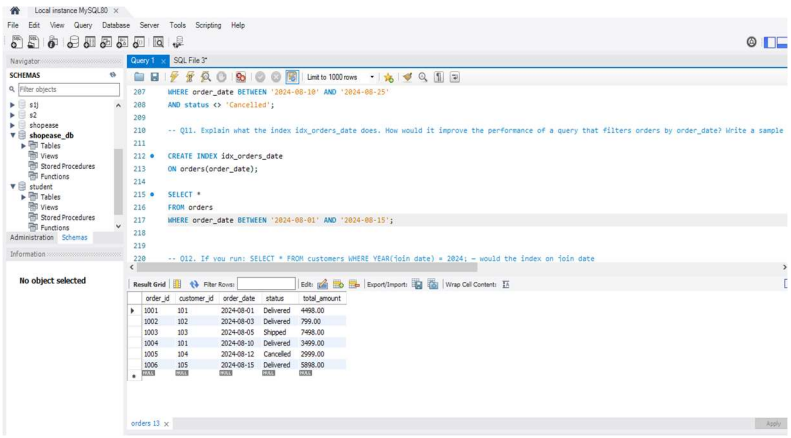
Q9. List all customers who joined in the year 2024 and belong to the state 'Maharashtra'.



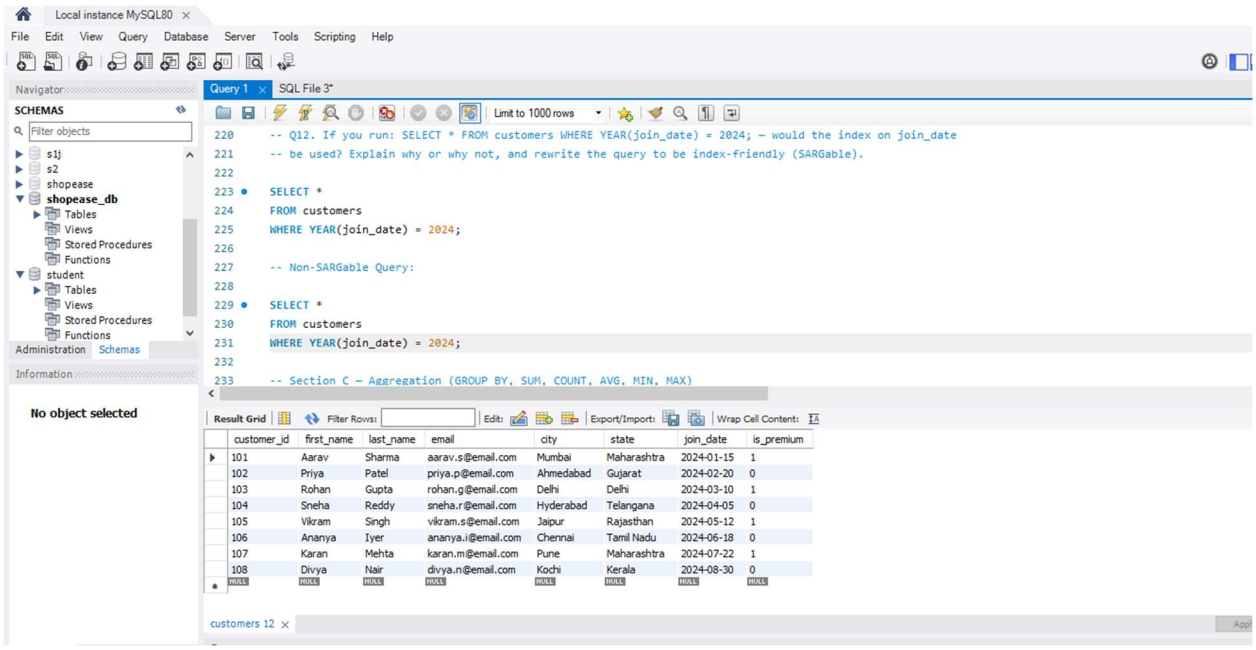
Q10. Find all orders placed between '2024-08-10' and '2024-08-25' (inclusive) that are NOT cancelled.



Q11. Explain what the index `idx_orders_date` does. How would it improve the performance of a query that filters orders by `order_date`? Write a sample query that would benefit from this index.

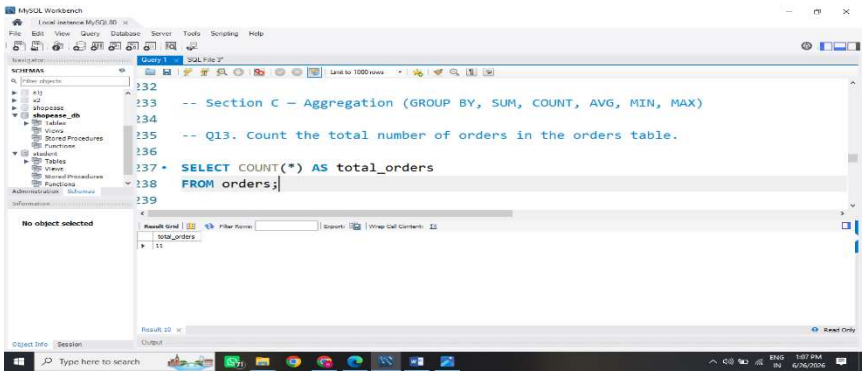


Q12. If you run: SELECT * FROM customers WHERE YEAR(join_date) = 2024; — would the index on join_date be used? Explain why or why not, and rewrite the query to be index-friendly (SARGable).

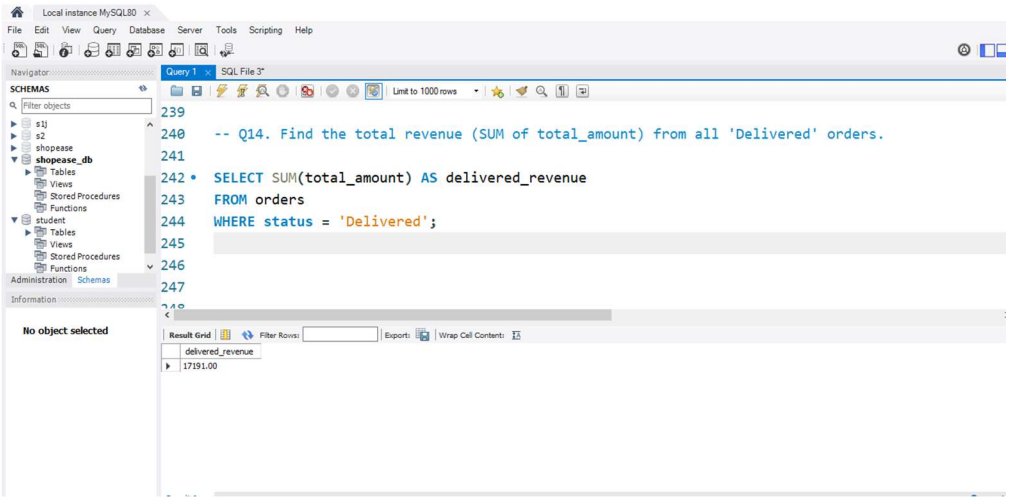


Section C — Aggregation (GROUP BY, SUM, COUNT, AVG, MIN, MAX)

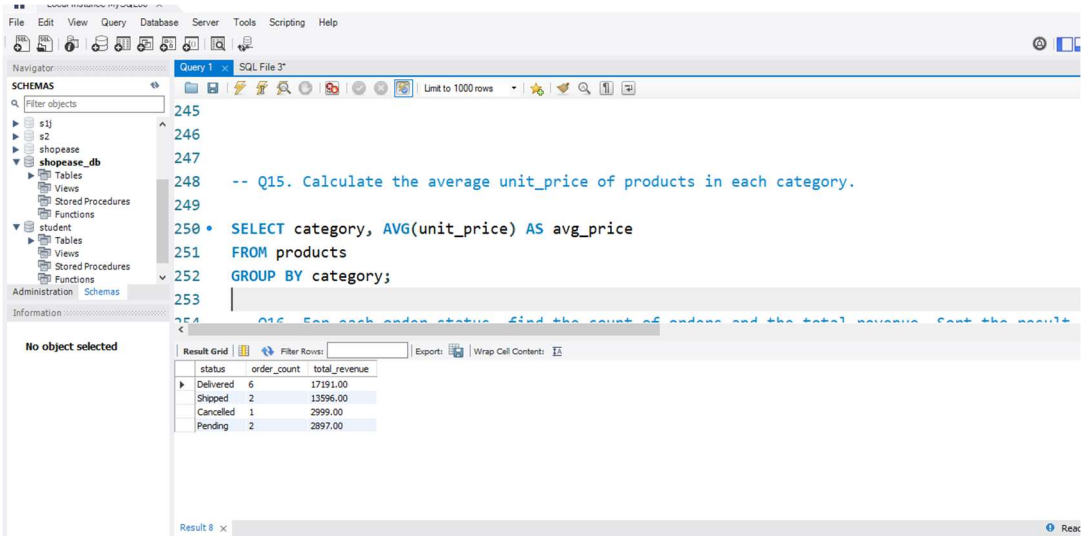
Q13. Count the total number of orders in the orders table.



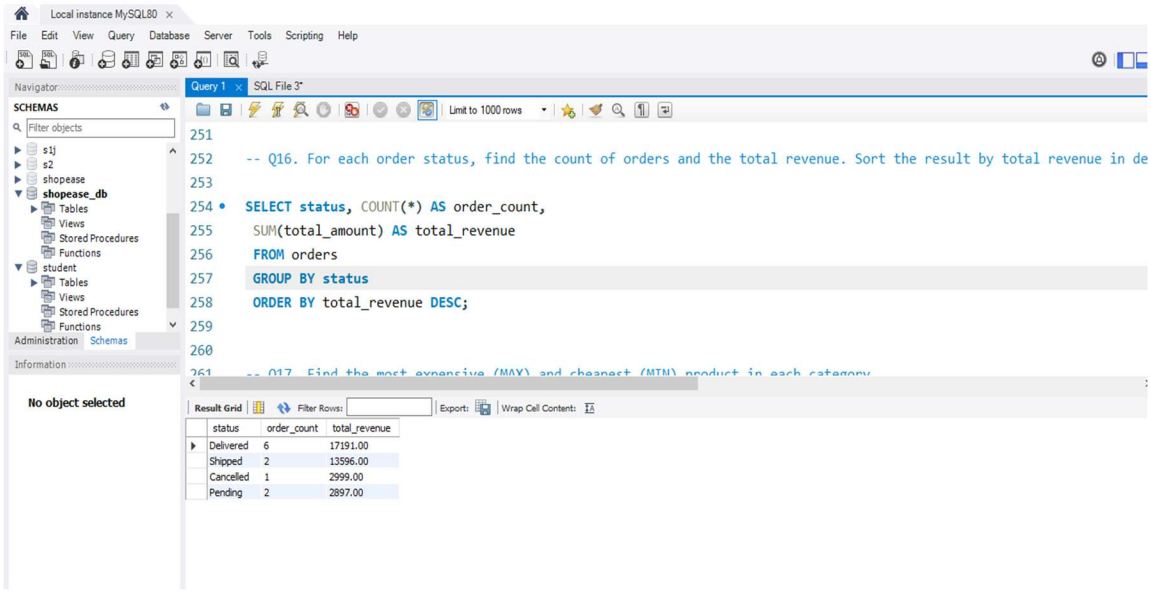
Q14. Find the total revenue (SUM of total_amount) from all 'Delivered' orders.



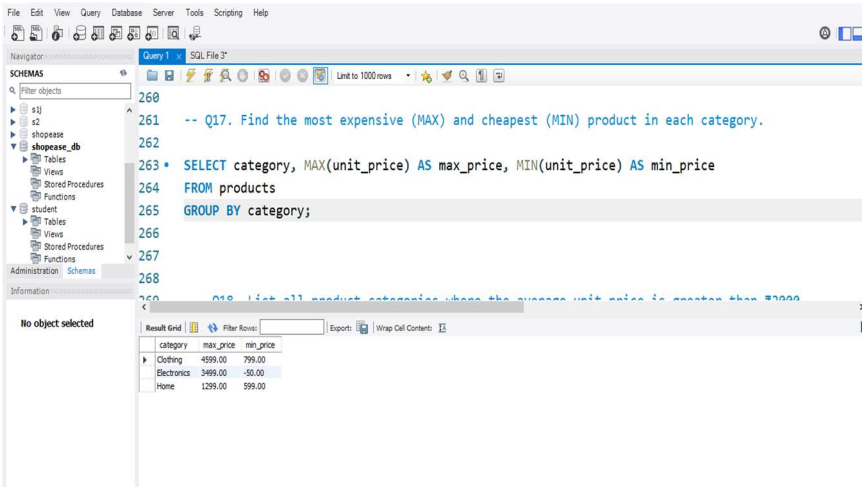
Q.15. Calculate the average unit_price of products in each category.



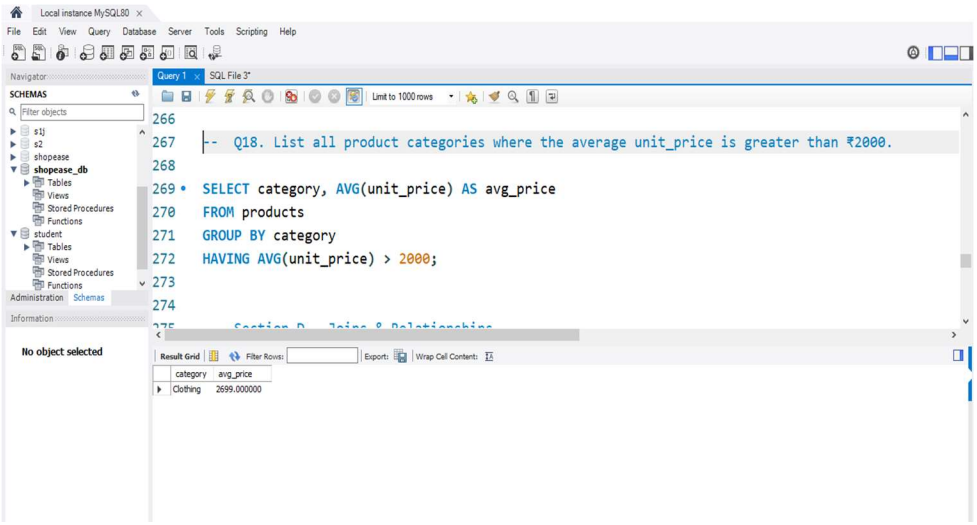
Q16. For each order status, find the count of orders and the total revenue. Sort the result by total revenue in descending order.



Q17. Find the most expensive (MAX) and cheapest (MIN) product in each category.

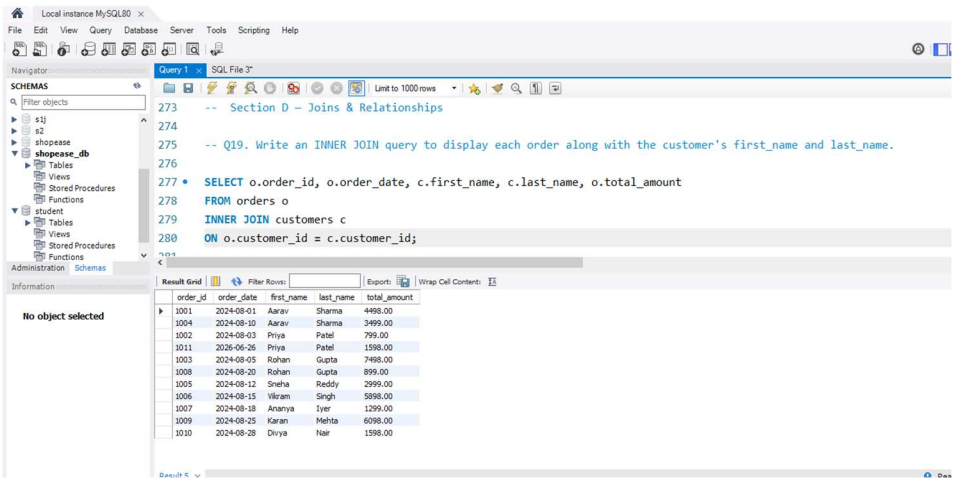


Q18. List all product categories where the average unit_price is greater than ₹2000.



Section D — Joins & Relationships

Q19. Write an INNER JOIN query to display each order along with the customer's first_name and last_name.



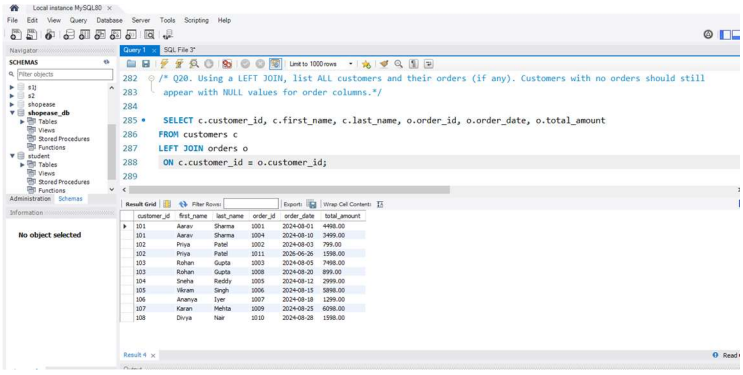
```
-- Section D — Joins & Relationships

-- Q19. Write an INNER JOIN query to display each order along with the customer's first_name and last_name.

SELECT o.order_id, o.order_date, c.first_name, c.last_name, o.total_amount
FROM orders o
INNER JOIN customers c
ON o.customer_id = c.customer_id;
```

order_id	order_date	first_name	last_name	total_amount
1001	2024-08-01	Aarav	Sharma	4498.00
1004	2024-08-10	Aarav	Sharma	3499.00
1002	2024-08-03	Priya	Patel	799.00
1011	2026-06-26	Priya	Patel	1598.00
1003	2024-08-05	Rohan	Gupta	7498.00
1008	2024-08-20	Rohan	Gupta	899.00
1005	2024-08-12	Sneha	Reddy	2999.00
1006	2024-08-15	Vikram	Singh	5898.00
1007	2024-08-18	Ananya	Iyer	1299.00
1009	2024-08-25	Karan	Mehta	6098.00
1010	2024-08-28	Divya	Nair	1598.00

Q20. Using a LEFT JOIN, list ALL customers and their orders (if any). Customers with no orders should still appear with NULL values for order columns.

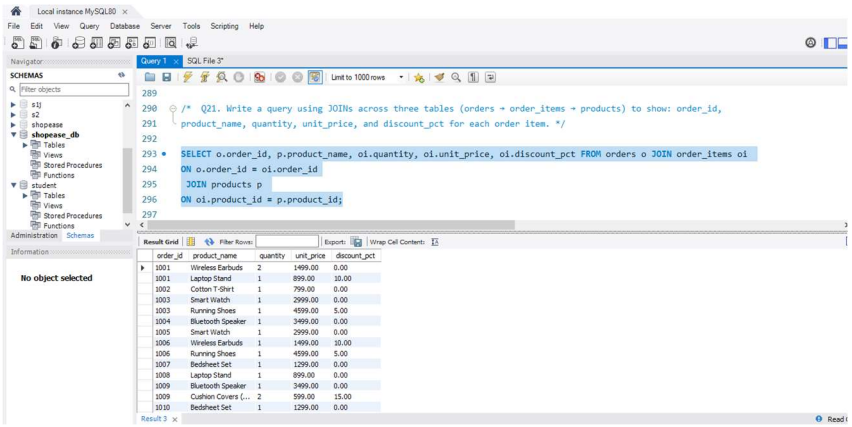


```
/* Q20. Using a LEFT JOIN, list ALL customers and their orders (if any). Customers with no orders should still
  appear with NULL values for order columns.*/

SELECT c.customer_id, c.first_name, c.last_name, o.order_id, o.order_date, o.total_amount
FROM customers c
LEFT JOIN orders o
ON c.customer_id = o.customer_id;
```

customer_id	first_name	last_name	order_id	order_date	total_amount
101	Aarav	Sharma	1001	2024-08-01	4498.00
101	Aarav	Sharma	1004	2024-08-10	3499.00
102	Priya	Patel	1002	2024-08-03	799.00
102	Priya	Patel	1011	2026-06-26	1598.00
103	Rohan	Gupta	1003	2024-08-05	7498.00
103	Rohan	Gupta	1008	2024-08-20	899.00
104	Sneha	Reddy	1005	2024-08-12	2999.00
105	Vikram	Singh	1006	2024-08-15	5898.00
106	Ananya	Iyer	1007	2024-08-18	1299.00
107	Karan	Mehta	1009	2024-08-25	6098.00
108	Divya	Nair	1010	2024-08-28	1598.00

Q21. Write a query using JOINs across three tables (orders → order_items → products) to show: order_id, product_name, quantity, unit_price, and discount_pct for each order item.



Q22. Explain the difference between LEFT JOIN and RIGHT JOIN with an example from this schema. When would you use a FULL OUTER JOIN?

LEFT JOIN:
Returns all rows from the left table and matching rows from the right table.

SELECT * FROM customers c
LEFT JOIN
orders o ON c.customer_id = o.customer_id;

RIGHT JOIN:
Returns all rows from the right table and matching rows from the left table.

SELECT * FROM customers c
RIGHT JOIN orders o
ON c.customer_id = o.customer_id;

FULL OUTER JOIN:
Returns all rows from both tables whether matches exist or not.

Use when you need:

- All customers
- All orders
- Including unmatched records from both tables

(MySQL does not directly support FULL OUTER JOIN.)

Q23. Identify all Foreign Key relationships in the schema.

Foreign Keys:

1. orders.customer_id → customers.customer_id

```
FOREIGN KEY(customer_id)
 REFERENCES customers(customer_id)
```

2. order_items.order_id → orders.order_id

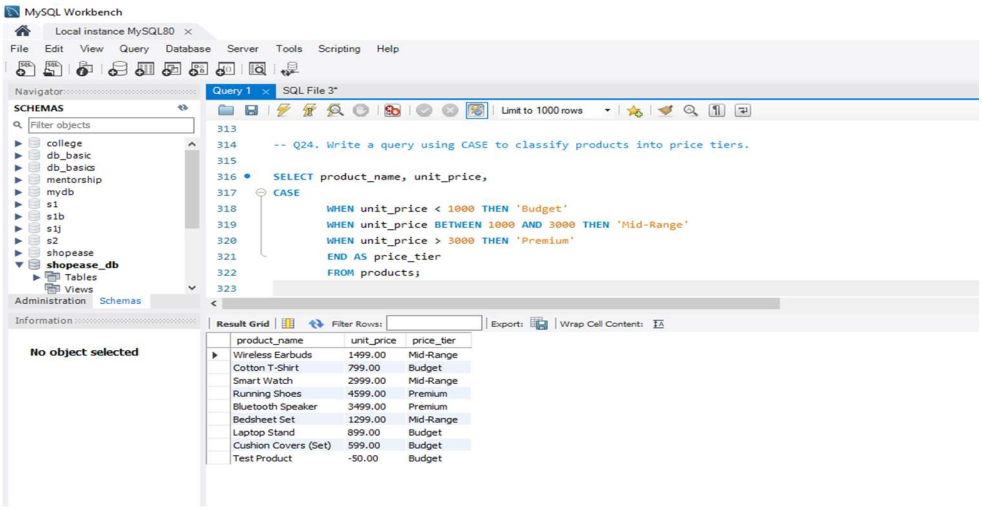
```
FOREIGN KEY(order_id)
 REFERENCES orders(order_id)
```

3. order_items.product_id → products.product_id

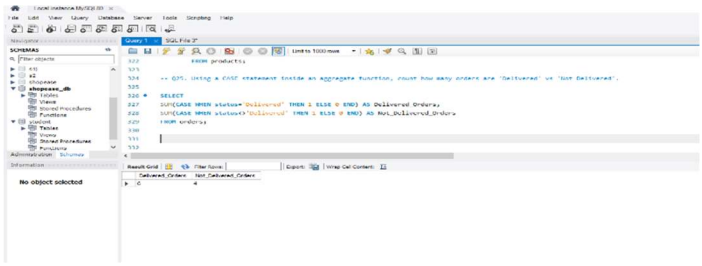
```
FOREIGN KEY(product_id)
REFERENCES products(product_id)
```

Section E — Advanced Concepts (CASE, ACID, Transactions)

Q24. Write a query using CASE to classify products into price tiers.



Q25. Using a CASE statement inside an aggregate function, count how many orders are 'Delivered' vs 'Not Delivered'.



Q26. Explain each letter of ACID.

A – Atomicity

Meaning**:** Either all steps of a transaction are completed successfully, or none of them are.
Example: If ₹5,000 is transferred from Account A to Account B and the system fails before the money reaches Account B, the entire transaction is cancelled (rolled back).

C – Consistency
Meaning: The database always remains correct and follows all rules before and after a transaction.
Example: An order cannot be placed for a customer who does not exist in the `customers` table.

I – Isolation
Meaning: Multiple transactions running at the same time do not affect each other.

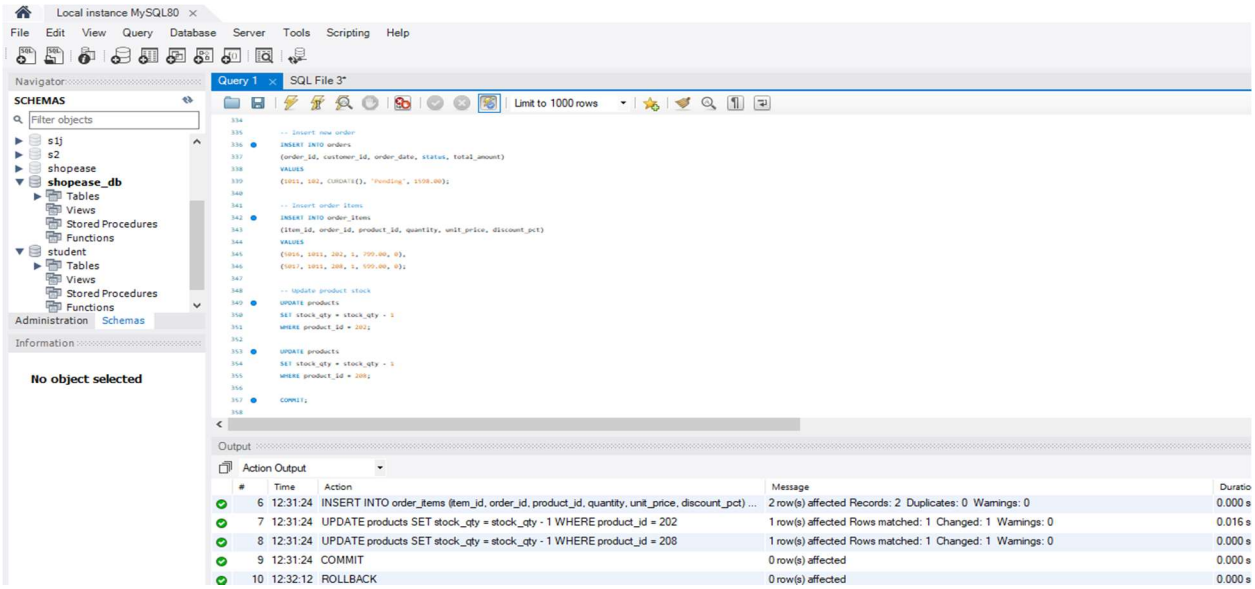
Example: If two customers buy the last product simultaneously, the database processes the transactions separately to avoid incorrect stock updates.

D – Durability
Meaning: Once a transaction is committed, the changes are saved permanently.

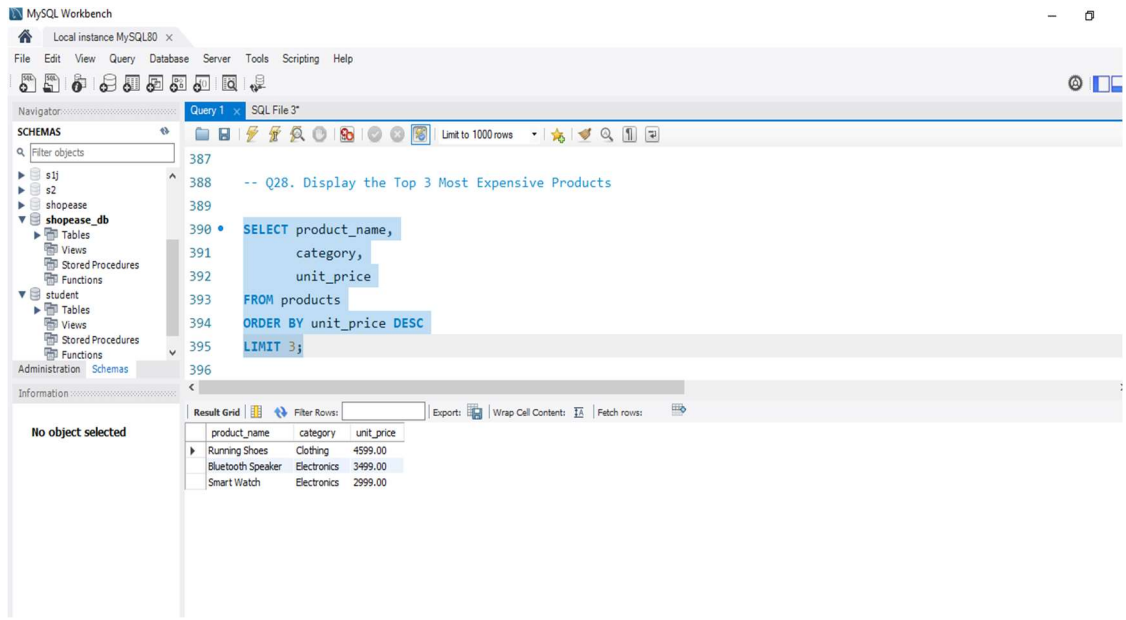
Example: After a successful online payment, the order remains saved even if the system crashes.

Importance: Ensures that committed data is never lost.

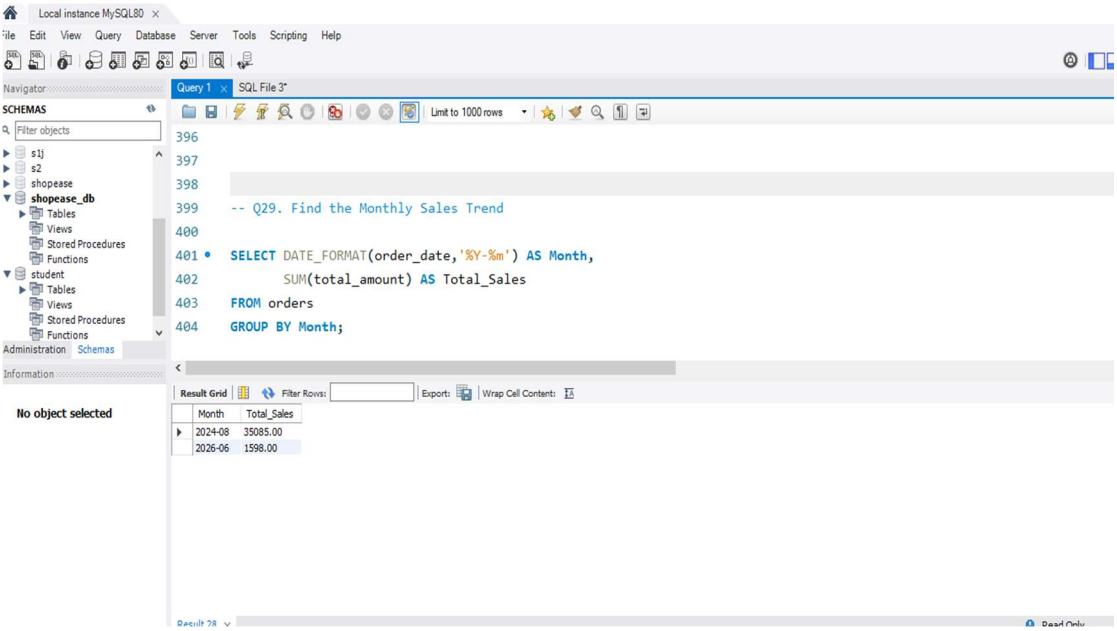
Q27. Write a SQL transaction that does the following atomically:



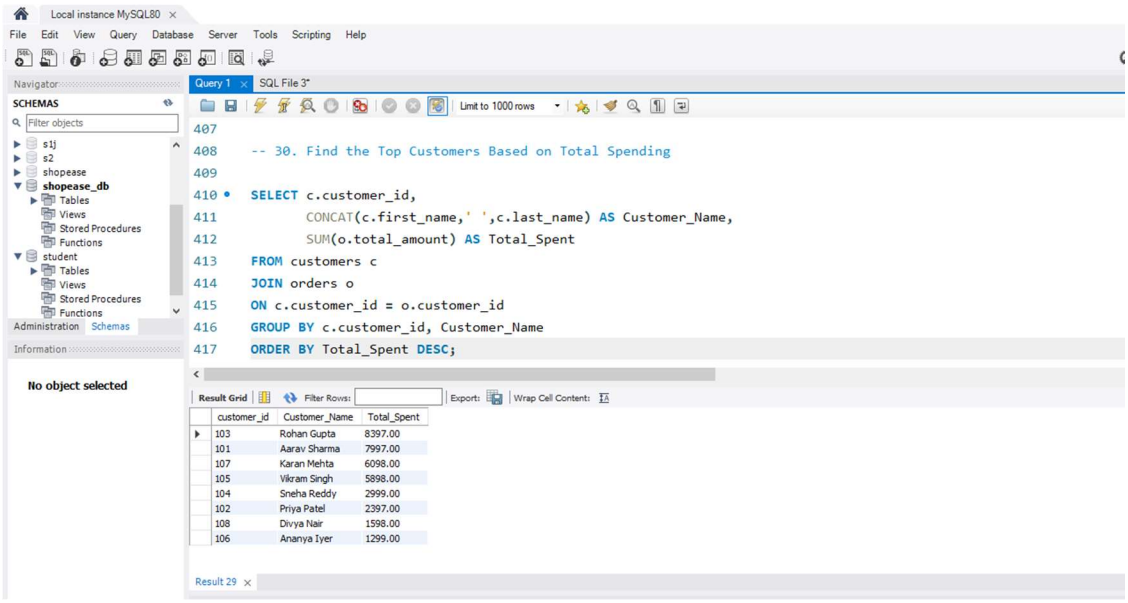
Q28. Display the Top 3 Most Expensive Products



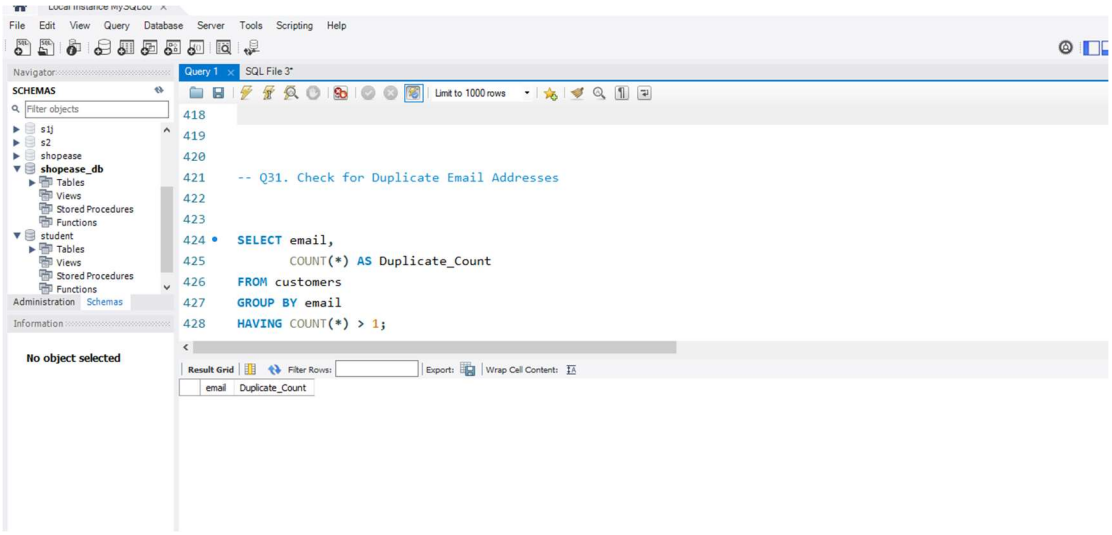
Q29. Find the Monthly Sales Trend



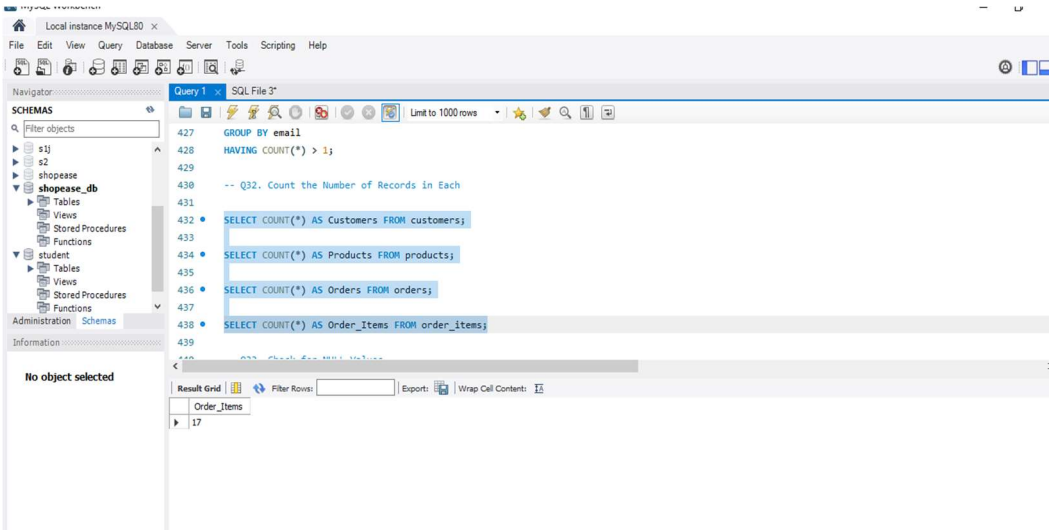
Q.30. Find the Top Customers Based on Total Spending



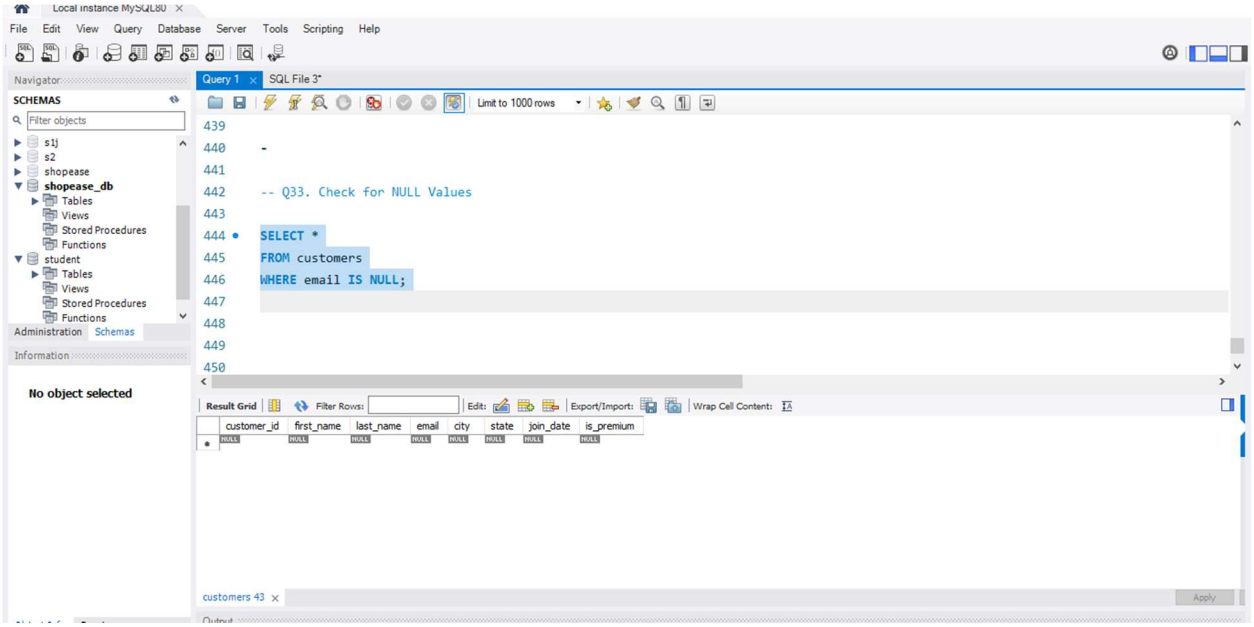
Q31. Check for Duplicate Email Addresses



Q32. Count the Number of Records in Each



Q33. Check for NULL Values



Brief Insights:

- The database consists of customer, product, order, and order item information stored in related tables.
- Delivered orders contribute the highest revenue among all order statuses.
- Electronics and Clothing products have the highest average selling prices.
- No duplicate email addresses were found due to the `UNIQUE` constraint.
- Aggregate functions such as `SUM()`, `COUNT()`, and `AVG()` provide valuable business summaries.
- JOIN operations combine customer, order, and product information effectively.
- Primary Keys and Foreign Keys maintain referential integrity.
- SQL queries help transform raw sales data into meaningful business insights for decision-making.

Conclusion:

This assignment successfully demonstrates the use of SQL for analyzing sales data using MySQL Workbench. The database was created, populated with sample data, and analyzed using filtering, aggregation, sorting, joins, CASE statements, and transactions. Business reports such as top customers, monthly sales trends, and product analysis were generated successfully. Data validation queries confirmed data integrity, while constraints and indexes ensured reliable and efficient database operations. Overall, the assignment shows how SQL can be effectively used for real-world business data analysis and decision-making.